

CAHIER DES CHARGES – Application Web “CampusMaster”

Plateforme pédagogique avancée pour étudiants de Master 2

1. Présentation du projet

1.1. Contexte

Dans le cadre du module “Projet Opérationnel”, les étudiants doivent réaliser une application ou un site web complet permettant la gestion, la diffusion et le suivi de contenus pédagogiques pour un Master 2.

L’objectif est de confronter les étudiants à un projet technique complet, incluant conception, architecture, développement, sécurité, qualité logicielle et déploiement.

1.2. Objectif général

Développer une plateforme web permettant :

- La mise à disposition de cours et supports pédagogiques
 - La gestion des étudiants et enseignants
 - La remise de devoirs
 - L’évaluation (notes, feedback)
 - La communication interne (annonces, messages)
 - Un tableau de bord avancé (statistiques et suivi)
-

2. Public cible

- Étudiants en Master 2
 - Enseignants / Encadrants
 - Administrateurs pédagogiques
-

3. Fonctionnalités (exigences fonctionnelles)

3.1. Gestion des utilisateurs

- Inscription / Connexion (email + mot de passe)
- Profils utilisateurs (étudiant, enseignant, admin)
- Gestion des rôles et permissions
- Réinitialisation du mot de passe

3.2. Espace Étudiant

- Consultation des cours
- Téléchargement des supports (PDF, PPT, vidéo)
- Dépôt de devoirs (upload + versionning)
- Consultation des notes et feedback
- Participation à des discussions / forums
- Notifications : nouveaux cours, deadlines, notes publiées

3.3. Espace Enseignant

- Ajout / modification / suppression de cours
- Mise en ligne des supports pédagogiques
- Création et gestion des devoirs (titre, consigne, date limite)
- Correction des devoirs (notation + commentaire)
- Publication d'annonces
- Gestion des étudiants (validation de profils, suivi)

3.4. Espace Administrateur

- Gestion des utilisateurs et rôles
- Gestion des modules, matières, semestres
- Modération des contenus
- Génération de statistiques avancées :
 - nombre d'étudiants actifs
 - taux de remise des devoirs
 - performance globale par matière

3.5. Tableau de bord analytique (complexité avancée)

- Graphe d'évolution des notes
- Activité hebdomadaire (pages vues, téléchargements)
- Statistiques par département / matière
- KPI configurables : taux d'assiduité, retards, réussite...

3.6. Module de messagerie interne

- Messages privés étudiant $\rightleftarrows$ enseignant
- Groupe par matière
- Système de tags (#urgent, #annonce, #projet)

3.7. Système de notifications push

- Email + notifications web real-time (WebSockets / WebPush)
- Déclenchement automatique selon événements :
 - devoir publié
 - devoir corrigé
 - nouveau message
 - deadline proche

4. Exigences techniques

4.1. Frontend

📌 Next.js 14 (App Router)

- SSR + SSG
- TailwindCSS ou Shadcn UI
- Gestion d'état : Zustand, Context API
- Upload de fichiers (via API ou storage cloud)
- Charts : Recharts / Chart.js

4.2. Backend

Technos recommandées (choix libre selon équipe) :

Option 1 – Java Spring Boot

✓ robuste, idéal pour un projet académique avancé

Option 2 – Laravel (si équipe à l'aise)

✓ rapides à mettre en place

✓ file d'attente, notifications, ACL

Backend attendu :

- Architecture RESTful (ou GraphQL si niveau avancé)
- API sécurisée (JWT + Refresh Token)
- WebSockets pour notifications en temps réel
- Rate limiting
- Documentation Swagger / OpenAPI

4.3. Base de données (SGBD)

Recommandé pour un projet complexe :

PostgreSQL

✓ performant

✓ intégrité forte

✓ JSONB possible

Alternatives acceptées :

- MySQL / MariaDB
- MongoDB (si choix d'architecture NoSQL)

4.4. Architecture logicielle

- Clean architecture
 - Séparation claire :
 - domain
 - application
 - infrastructure
 - presentation
 - CI/CD (GitHub Actions ou GitLab CI)
-

5. Exigences non fonctionnelles

5.1. Performance

- Caching (Redis)

5.2. Sécurité

- Hashage des mots de passe (bcrypt)
- Protection CSRF / XSS / SQL Injection
- Permissions RBAC
- Audit log (actions sensibles enregistrées)

5.3. Qualité logicielle

- Tests unitaires (70% min)
- Tests d'intégration API
- Vérification des types (TypeScript obligatoire côté frontend + si backend Node)

5.4. Accessibilité

- Mode sombre et light
-

6. Livrables attendus

- Diagrammes UML :
 - cas d'usage
 - classes
 - séquence
- Maquettes Figma
- Documentation technique complète
- Dépôt Git avec branches organisées
- API documentée (Swagger / Postman)
- Rapport final + code source en zip

7. Déploiement

- Backend :
 - VPS (Ubuntu 22.04) ou Railway / Render
- Frontend : Vercel ou serveur Dockerisé
- Base de données : PostgreSQL

9. Critères d'évaluation

- Pertinence de l'architecture
- Qualité du code et tests
- Respect du cahier des charges
- Fonctionnalités réalisées
- UX / UI
- Documentation
- Déploiement final
- Capacité à travailler en équipe et à présenter